Real-Time Object Detection Using YOLOv8 for Mac Webcam Computer Vision Applications

Dominique McClaney
CS 7367 - Machine Vision
Kennesaw State University College of Computing and Software Engineering
Georgia Gwinnett College, B.S. Software Development

Abstract—This project implements and evaluates a real-time object detection system using the You Only Look Once version 8 nano model (YOLOv8n), a Mac integrated webcam, Python, OpenCV, and the Ultralytics framework. The work expands an approved proposal focused on YOLO for practical computer vision applications. A pretrained COCO model was used to identify and localize objects in a live 1280 x 720 video stream. The application overlays class names, confidence scores, bounding boxes, frames per second (FPS), inference time, detection count, and brightness. It also records frame-level metrics to CSV and saves annotated evidence images. Across 7,114 logged frames, the system averaged 16.70 FPS with a median of 14.94 FPS. Mean inference time was 23.52 ms, and detections occurred in 52.5% of frames. The detector recognized people, bottles, cell phones, clocks, oranges, potted plants, vases, and a remote/controller. A lime was misclassified as a sports ball, illustrating the limits of using a general-purpose pretrained model without domain-specific fine-tuning. The results show that a consumer Mac can provide useful near-real-time YOLO inference and a repeatable platform for computer vision experimentation.

Keywords—computer vision, object detection, YOLOv8, OpenCV, real-time inference, webcam, COCO.

I. INTRODUCTION

Object detection identifies both what appears in an image and where each object is located. It supports autonomous systems, robotics, surveillance, manufacturing, healthcare, smart-city systems, and assistive technology. Earlier detection pipelines often separated region proposal, feature extraction, and classification into multiple stages. YOLO instead predicts bounding boxes and class probabilities in one network evaluation, allowing the complete pipeline to be optimized end to end [1].

This final project follows the approved proposal title and objective, Real-Time Object Detection Using YOLO for Computer Vision Applications. The original proposal planned to investigate YOLO accuracy and computational efficiency with public datasets and benchmark metrics. The completed implementation narrows that goal to a reproducible live-webcam experiment on a Mac. It uses a pretrained YOLOv8n model and measures runtime behavior while testing multiple everyday objects.

The project goals were to: (1) create a working real-time detector; (2) display bounding boxes, labels, and confidence values; (3) record FPS and inference time; (4) save annotated evidence; (5) identify both successful and unsuccessful predictions; and (6) document the system in an IEEE-style technical paper.

II. RELATED WORK

The original YOLO paper framed detection as direct regression from a full image to bounding boxes and class probabilities. The base system was reported at 45 FPS, while Fast YOLO reached 155 FPS, demonstrating the speed advantage of a unified detector [1]. YOLO9000 extended the approach to thousands of object categories through joint detection and classification training [2].

YOLOv3 introduced multi-scale prediction and improved accuracy while retaining high speed. The authors reported 22-ms processing at 320 x 320 and strong performance under the older AP50 metric [3]. YOLOv4 combined CSP connections, mosaic augmentation, CIoU loss, and other training strategies, reporting 43.5% AP on COCO at approximately 65 FPS on a Tesla V100 [4].

The implementation in this project uses Ultralytics YOLOv8n. The nano model is appropriate for a local webcam demonstration because it emphasizes low computational cost. Unlike a custom-trained detector, it uses pretrained COCO categories. This allows immediate inference but can also cause class confusion when an object is visually similar to a COCO category.

III. SYSTEM DESIGN AND ARCHITECTURE

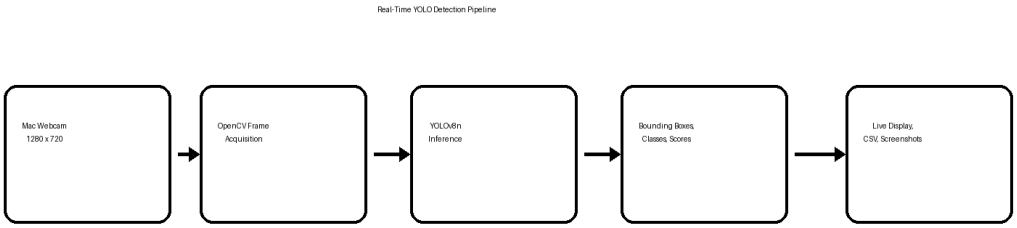

Fig. 1. End-to-end Mac webcam YOLOv8n detection architecture.

Figure 1 shows the project architecture. OpenCV opens camera index 0 and requests a 1280 x 720 frame. The program sends each frame to the YOLOv8n model using a confidence threshold of 0.35. The model returns bounding boxes, class identifiers, and confidence scores. Ultralytics renders the annotations, and the application adds its own FPS, inference-time, object-count, and brightness telemetry before displaying the frame.

A. Data Flow

The processing loop reads a frame, records the inference start time, executes model prediction, measures elapsed inference time, and draws model output. FPS is calculated from the interval between successive processed frames. Brightness is the mean intensity of a grayscale conversion. Frame number, FPS, inference time, detection count, average confidence, and brightness are appended to runtime_metrics.csv.

B. Output Management

The project creates an outputs directory and a screenshots subdirectory. The user can save an annotated image, start or stop an MP4 recording, and quit. The eight retained screenshots preserve their original names so that the code outputs, report figures, and submission folder remain easy to cross-reference.

IV. IMPLEMENTATION

The implementation was completed in Python using OpenCV for camera access and display, Ultralytics for YOLO inference, pathlib for portable paths, csv for telemetry storage, and time.perf_counter for high-resolution timing. The model file is yolov8n.pt. The confidence threshold is 0.35, and the requested capture size is 1280 x 720.

Parameter	Value
Platform	Mac integrated webcam
Language	Python
Model	YOLOv8n pretrained on COCO
Frame request	1280 x 720
Confidence threshold	0.35
Outputs	Live display, CSV, PNG, optional MP4
Logged frames	7,114

TABLE I. IMPLEMENTATION CONFIGURATION

The detector was tested with people and common household objects. The retained evidence includes a bottle, cell phone, clock, orange, person, potted plant, vase, controller/remote, and a lime. Testing deliberately included ordinary and ambiguous objects to show both strengths and limitations.

V. EXPERIMENTAL METHOD

The live application was run from a Python virtual environment on the Mac. The webcam observed the subject and one or more presented objects. Each frame was processed independently. Screenshots were saved after a stable detection appeared. The program logged 7,114 frames, enabling analysis of speed, inference latency, detections, confidence, and scene brightness.

The evaluation is a deployment-oriented functional study rather than a formal COCO validation run. Consequently, the report does not claim a new mAP, precision, or recall benchmark. Instead, it reports measured runtime performance and manually reviews selected detections. This distinction prevents webcam observations from being incorrectly presented as a labeled benchmark test set.

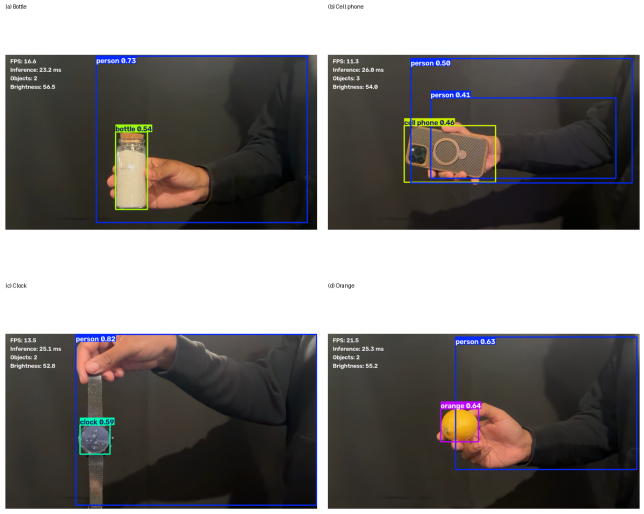

Fig. 2. Successful detections: bottle, cell phone, clock, and orange. Original filenames are preserved in the submission.

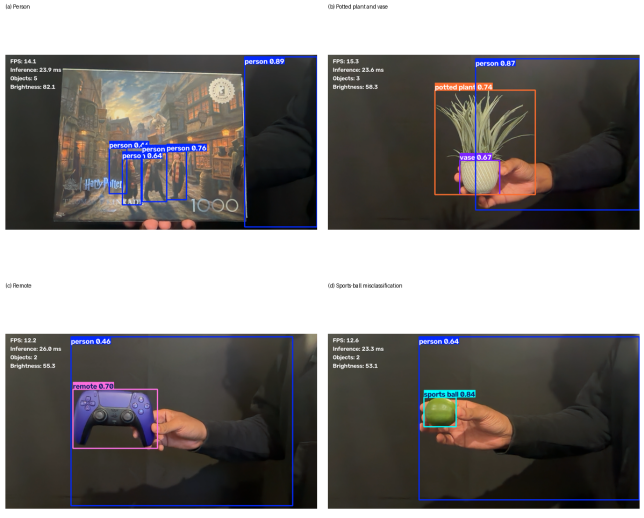

Fig. 3. Person, plant/vase, remote, and sports-ball misclassification examples.

VI. RESULTS

Metric	Measured value
Frames analyzed	7,114
Average FPS	16.70
Median FPS	14.94
Observed FPS range	1.42 - 28.41
Mean inference time	23.52 ms
Median inference time	23.48 ms
95th percentile inference	26.07 ms
Frames with detection	52.5%
Mean confidence, detection frames	0.626
Maximum objects in one frame	7

TABLE II. RUNTIME RESULTS FROM RUNTIME_METRICS.CSV

The detector averaged 16.70 FPS and 23.52 ms of model inference per frame. End-to-end FPS is lower than the reciprocal of inference time because the application also performs capture, plotting, text overlay, CSV writing, GUI display, and timing. At least one object was detected in 52.5% of logged frames.

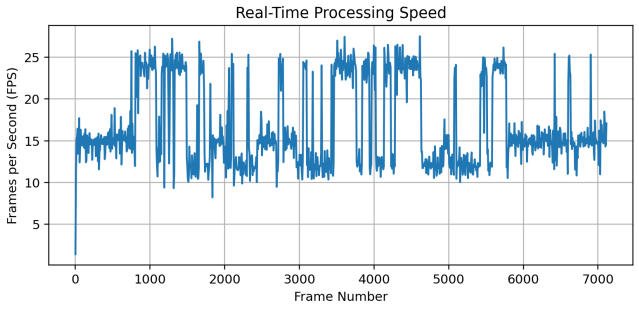

Fig. 4. Measured end-to-end FPS over the recorded session.

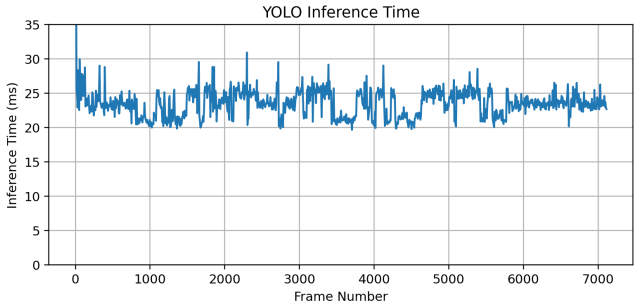

Fig. 5. YOLOv8n inference time over the recorded session.

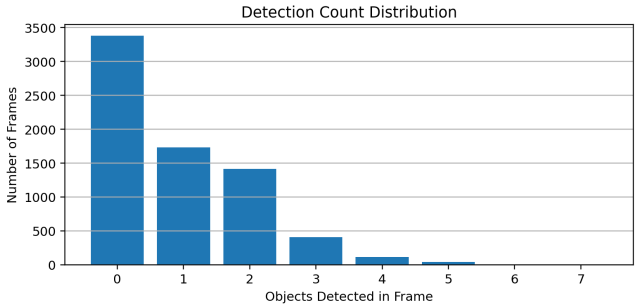

Fig. 6. Distribution of object counts across logged frames.

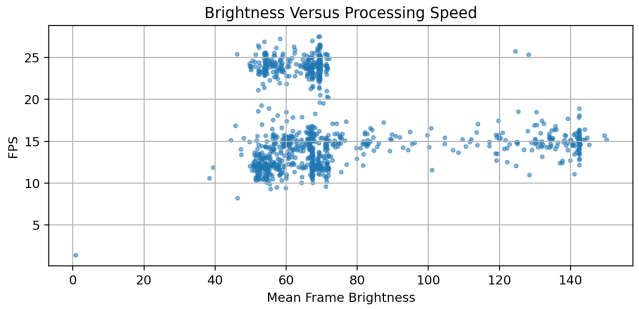

Fig. 7. Relationship between mean frame brightness and FPS.

The selected screenshots demonstrate multiple correct detections. The plant frame is especially useful because the model simultaneously recognized the complete object as a potted plant, the lower container as a vase, and the subject as a person. The puzzle image also produced multiple person detections from printed artwork, showing that the model can recognize learned visual patterns even when the subjects are not physically present.

VII. QUALITATIVE ERROR ANALYSIS

The sports-ball image is an intentional error example. A green lime was classified as a sports ball with confidence 0.84. The prediction is understandable because the object is round, similarly sized within the frame, and lacks contextual cues that clearly separate fruit from a ball. This result illustrates that confidence is not the same as correctness.

A controller was identified as a remote, which can be treated as a semantically reasonable COCO-class mapping rather than a precise product classification. The watch was labeled as a clock, also reflecting the available pretrained categories. These examples demonstrate how a general dataset influences the vocabulary and granularity of model output.

VIII. DISCUSSION

The completed system met the core proposal objective by implementing practical real-time YOLO object detection. It localized multiple objects, assigned labels and confidence scores, and maintained useful interactive speed on a consumer Mac. The telemetry and saved outputs made the demonstration repeatable and provided stronger evidence than a visual-only application.

The project also reinforced the difference between a pretrained demonstration and a custom research model. No new model weights were trained in this phase. Therefore, the detector inherits COCO strengths, category definitions, and biases. A custom dataset containing the specific household objects used in the experiment could support fine-tuning and a formal train/validation/test evaluation.

The measured average of approximately 16.7 FPS was sufficient for responsive webcam interaction. Inference itself averaged about 23.5 ms, but total throughput included non-model overhead. Future optimization could buffer CSV writes, reduce display resolution, use hardware acceleration, or process every second frame.

IX. LIMITATIONS AND FUTURE WORK

The primary limitation is the absence of a labeled validation set for these webcam scenes. The project cannot honestly report precision, recall, or mAP from the eight screenshots alone. The raw log records average confidence rather than per-class ground truth. Lighting, distance, framing, and motion were not systematically controlled.

Future work will create a labeled dataset, fine-tune a small YOLO model, calculate class-specific precision, recall, F1 score, and mAP, and compare CPU and accelerated inference. The system can later be deployed to a Raspberry Pi 5 and Sony IMX500 camera as an edge-AI extension. Additional applications may include assistive technology, public safety, intelligent transportation, robotics, and healthcare.

These directions align with the author's academic background in software development and artificial intelligence and the long-term goal of building computer vision systems that improve safety, access, and quality of life.

X. CONCLUSION

This project designed, implemented, and evaluated a real-time YOLOv8n detection application using a Mac webcam. The application successfully detected multiple COCO object classes, logged 7,114 frames, and achieved an average of 16.70 FPS with 23.52 ms mean inference time. The preserved evidence images show correct detections as well as meaningful failure cases. The results support YOLO as a practical starting point for real-time computer vision while also showing why domain-specific data and formal evaluation are necessary before deployment in high-stakes systems.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE CVPR, 2016, pp. 779-788.

[2] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in Proc. IEEE CVPR, 2017, pp. 7263-7271.

[3] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv:1804.02767, 2018.

[4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” arXiv:2004.10934, 2020.

[5] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. ECCV, 2014, pp. 740-755.

[6] Ultralytics, “Ultralytics YOLO Documentation,” software documentation, accessed July 2026.

[7] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.

[8] R. Girshick, “Fast R-CNN,” in Proc. IEEE ICCV, 2015, pp. 1440-1448.

[9] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” IEEE TPAMI, vol. 39, no. 6, pp. 1137-1149, 2017.

[10] W. Liu et al., “SSD: Single Shot MultiBox Detector,” in Proc. ECCV, 2016, pp. 21-37.

APPENDIX A. SUBMISSION ARTIFACTS

The code ZIP includes mac_yolo_detection.py, requirements.txt, run_mac.sh, runtime_metrics.csv, runtime_summary.csv, all eight screenshots with their original filenames, generated performance graphs, and this report in Word and PDF formats. The pretrained yolov8n.pt file is intentionally not included because Ultralytics downloads it automatically and model files increase archive size.

APPENDIX B. SOURCE CODE PRINTOUT

The complete executable source file is reproduced below and is also included as mac_yolo_detection.py in the Code folder of the submission ZIP.

```
from __future__ import annotations

import csv
import time
from datetime import datetime
from pathlib import Path

import cv2
from ultralytics import YOLO


#
============================================================
# Configuration
#
============================================================

MODEL_NAME = "yolov8n.pt"
CAMERA_INDEX = 0
CONFIDENCE_THRESHOLD = 0.35
FRAME_WIDTH = 1280
FRAME_HEIGHT = 720

PROJECT_FOLDER = Path(__file__).resolve().parent
OUTPUT_FOLDER = PROJECT_FOLDER / "outputs"
SCREENSHOT_FOLDER = OUTPUT_FOLDER / "screenshots"
METRICS_FILE = OUTPUT_FOLDER / "runtime_metrics.csv"
VIDEO_FILE = OUTPUT_FOLDER / "detection_recording.mp4"


def create_output_folders() -> None:
    OUTPUT_FOLDER.mkdir(exist_ok=True)
    SCREENSHOT_FOLDER.mkdir(exist_ok=True)


def initialize_metrics_file() -> None:
    if METRICS_FILE.exists():
        return

    with METRICS_FILE.open("w", newline="", encoding="utf-
8") as csv_file:
        writer = csv.writer(csv_file)

        writer.writerow(
            [
                "timestamp",
                "frame_number",
                "fps",
                "inference_ms",
                "detection_count",
                "average_confidence",
                "brightness",
            ]
        )


def calculate_brightness(frame) -> float:
    grayscale = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    return float(grayscale.mean())


def save_metrics(
    frame_number: int,
    fps: float,
    inference_ms: float,
    detection_count: int,
    average_confidence: float,
    brightness: float,
) -> None:
    with METRICS_FILE.open("a", newline="", encoding="utf-
8") as csv_file:
        writer = csv.writer(csv_file)

        writer.writerow(
            [

datetime.now().isoformat(timespec="seconds"),
                frame_number,
                round(fps, 3),
                round(inference_ms, 3),
                detection_count,
                round(average_confidence, 4),
                round(brightness, 3),
            ]
        )


def main() -> None:
    create_output_folders()
    initialize_metrics_file()

    print("Loading YOLO model...")
    model = YOLO(MODEL_NAME)

    camera = cv2.VideoCapture(CAMERA_INDEX)

    if not camera.isOpened():
        raise RuntimeError(
            "The Mac camera could not be opened. "
            "Check macOS camera permissions and close other
camera applications."
        )

    camera.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_WIDTH)
    camera.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_HEIGHT)

    actual_width =
int(camera.get(cv2.CAP_PROP_FRAME_WIDTH))
    actual_height =
int(camera.get(cv2.CAP_PROP_FRAME_HEIGHT))

    video_writer = None
    recording = False
    frame_number = 0
    previous_time = time.perf_counter()

    print("\nYOLO Mac Webcam Detection")
    print("--------------------------------")
    print("Q = Quit")
    print("S = Save screenshot")
    print("R = Start/stop video recording")
    print("--------------------------------\n")

    try:
        while True:
            success, frame = camera.read()

            if not success:
                print("Unable to read a camera frame.")
                break

            frame_number += 1

            inference_start = time.perf_counter()

            results = model.predict(
                source=frame,
                conf=CONFIDENCE_THRESHOLD,
                verbose=False,
            )

            inference_ms = (
                time.perf_counter() - inference_start
            ) * 1000

            annotated_frame = results[0].plot()

            current_time = time.perf_counter()
            elapsed = current_time - previous_time
            previous_time = current_time

            fps = 1.0 / elapsed if elapsed > 0 else 0.0

            boxes = results[0].boxes

            if boxes is not None and len(boxes) > 0:
                detection_count = len(boxes)
                confidence_values =
boxes.conf.cpu().numpy()
                average_confidence =
float(confidence_values.mean())
            else:
                detection_count = 0
                average_confidence = 0.0

            brightness = calculate_brightness(frame)

            cv2.putText(
                annotated_frame,
                f"FPS: {fps:.1f}",
                (20, 35),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255, 255, 255),
                2,
            )

            cv2.putText(
                annotated_frame,
                f"Inference: {inference_ms:.1f} ms",
                (20, 70),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255, 255, 255),
                2,
            )

            cv2.putText(
                annotated_frame,
                f"Objects: {detection_count}",
                (20, 105),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255, 255, 255),
                2,
            )

            cv2.putText(
                annotated_frame,
                f"Brightness: {brightness:.1f}",
                (20, 140),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255, 255, 255),
                2,
            )

            if recording:
                cv2.putText(
                    annotated_frame,
                    "RECORDING",
                    (actual_width - 190, 35),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.8,
                    (0, 0, 255),
                    2,
```

```
                )

                if video_writer is not None:
                    video_writer.write(annotated_frame)

            save_metrics(
                frame_number=frame_number,
                fps=fps,
                inference_ms=inference_ms,
                detection_count=detection_count,
                average_confidence=average_confidence,
                brightness=brightness,
            )

            cv2.imshow(
                "YOLO Real-Time Object Detection -
Dominique McClaney",
                annotated_frame,
            )

            key = cv2.waitKey(1) & 0xFF

            if key == ord("q"):
                break

            if key == ord("s"):
                timestamp =
datetime.now().strftime("%Y%m%d_%H%M%S")

                screenshot_path = (
                    SCREENSHOT_FOLDER
                    / f"yolo_detection_{timestamp}.png"
                )

                cv2.imwrite(
                    str(screenshot_path),
                    annotated_frame,
                )

                print(f"Screenshot saved:
{screenshot_path}")

            if key == ord("r"):
                recording = not recording

                if recording:
                    codec = cv2.VideoWriter_fourcc(*"mp4v")

                    video_writer = cv2.VideoWriter(
                        str(VIDEO_FILE),
                        codec,
                        20.0,
                        (actual_width, actual_height),
                    )

                    print(f"Recording started:
{VIDEO_FILE}")

                else:
                    if video_writer is not None:
                        video_writer.release()
                        video_writer = None

                    print("Recording stopped.")

    finally:
        camera.release()

        if video_writer is not None:
            video_writer.release()

        cv2.destroyAllWindows()

        print("\nYOLO detection stopped.")
        print(f"Metrics saved to: {METRICS_FILE}")
        print(f"Screenshots saved to: {SCREENSHOT_FOLDER}")


if __name__ == "__main__":
    main()
```